Image Processing - Exercise 2

Shay Morad

Introduction

The objective of this assignment was to complete 3 tasks involving audio files manipulation using FT, STFT and Spectrograms. Task 1 involved creating different versions of watermarks for an audio file. In Task 2, we've been requested to classify 9 audio files into 3 categories based on watermarks that were put inside of them, and find the function that describes each category/watermark visually (not exactly but approximately). To finish it all in Task 3, we were provided with 2 audio files, of which both were sped up/slowdown but in different domains, Time/Frequency, of the same original file, and we needed to find out which domain and methods were used on each of the files.

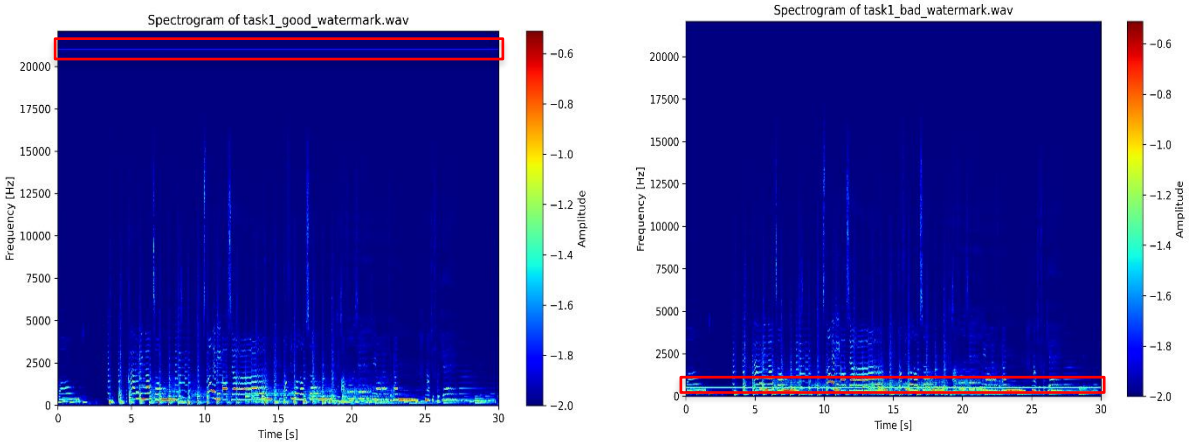
Adding Watermarks

In this task, the goal was to add watermarks to audio files using sine waves at specific frequencies. The exercise required the creation of two watermarks: one “good” watermark, which should be subtle and not interfere significantly with the original audio, and one “bad” watermark, which is louder and more noticeable. The watermarks were created using sine waves at different frequencies, with the amplitude and frequency of each watermark being key to how it affects the audio signal.

To achieve this, I used two different sine wave frequencies:

- **Good Watermark:** A sine wave at a high frequency of approximately 21 kHz, which is near the upper limit of human hearing. The amplitude of this watermark was kept low to minimize its perceptibility in the audio.

- **Bad Watermark:** A sine wave at a much lower frequency of 500 Hz, with a significantly higher amplitude. This watermark is easily audible and likely to interfere with the original audio signal, making it a “bad” watermark.



Spectrogram Analysis:

- **Good Watermark:** As expected, the good watermark is located around 21 kHz in the spectrogram, with a low amplitude. This watermark does not interfere much with the original audio and is mostly imperceptible to human listeners.
- **Bad Watermark:** The bad watermark, located at 500 Hz, has a much higher amplitude, making it easily detectable in the spectrogram. The frequency is clearly audible and distorts the original audio, making it an undesirable choice for watermarking.

Bad Watermark's Impact:

The bad watermark’s high amplitude and low frequency make it audible and clearly detectable in the audio. The perceptibility of this watermark makes it unsuitable for use in most audio watermarking applications, as it would degrade the quality of the original audio. On the other hand, the good watermark, due to its high frequency and low amplitude, is barely noticeable and does not compromise the audio's integrity.

Watermark Design for Images:

When designing a watermark for images, similar considerations should be taken into account:

Frequency Selection: The watermark should ideally use frequencies that are outside the range of human perception, so they do not interfere with the main content.

Amplitude Control: The watermark’s amplitude should be low enough not to affect the visual content of the image, yet still be detectable if needed.

Robustness: The watermark should remain detectable even after common image manipulations, such as resizing or compression.

Classifying watermarks

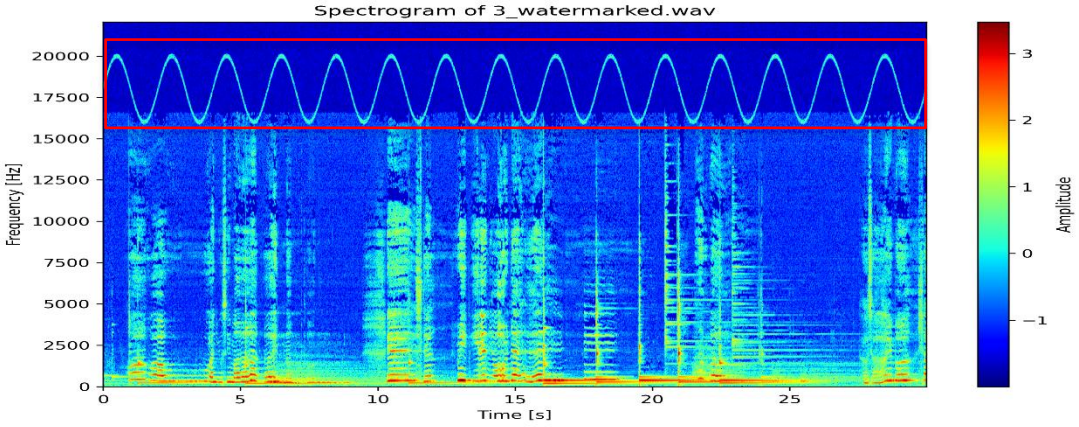
In this task, the goal was to detect and classify the watermarks embedded in several audio files. The watermark detection process aimed to identify and extract the embedded watermark signal from the audio, allowing for classification of the watermarked files.

Overall Approach for Detecting the Watermarks

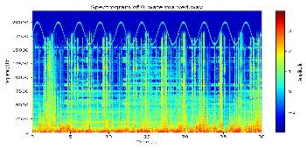
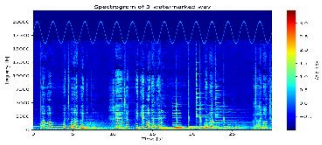
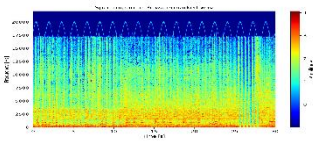
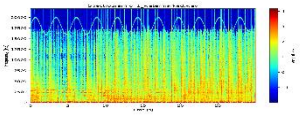
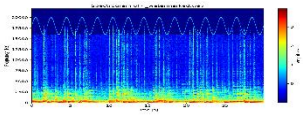
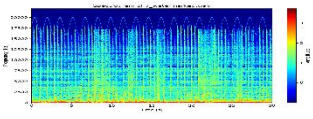
To classify the watermarks, I used a frequency analysis approach, relying on the fact that the watermarking frequencies are distinct from the main audio content. The steps for detecting the watermarks were as follows:

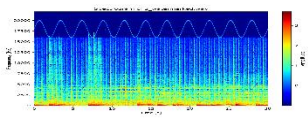
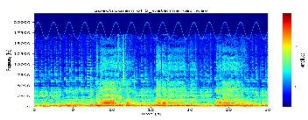
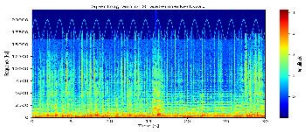
- Create a spectrogram for each audio file.
- Look for a visible watermark in each audio file.

Using this approach was very successful, as the watermarks were very visible as shown in the following spectrogram of one of the audio files. It is clear there's a watermarks that looks like the graph of a sine wave, between the frequencies 16kHz-20.2kHz.

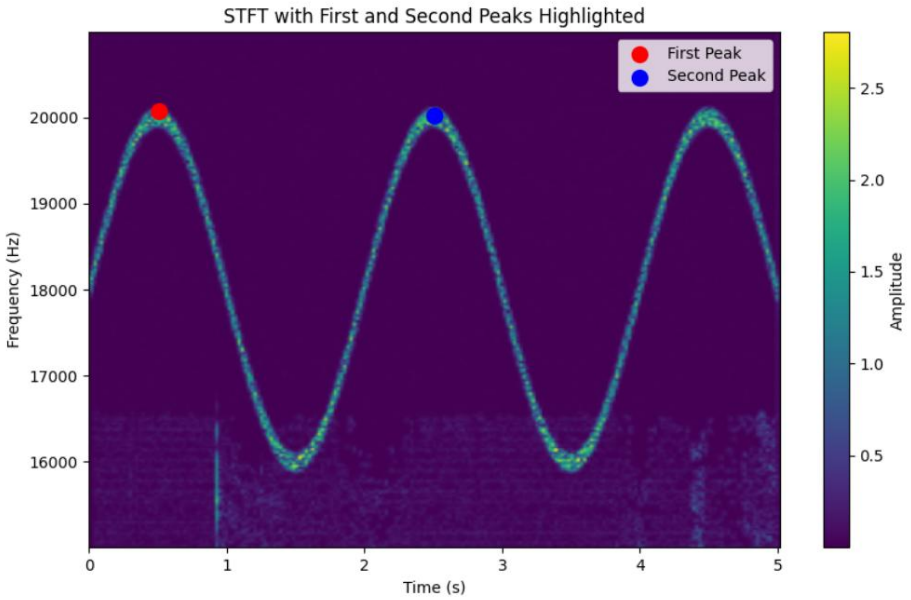


This is the grouping of the different audio signals with their watermarks being visible in their spectrograms, between the frequencies of 16kHz and 20.2kHz:

Category 1	Category 2	Category 3
Audio File 0 Spectrogram 	Audio File 3 Spectrogram 	Audio File 6 Spectrogram 
Audio File 1 Spectrogram 	Audio File 4 Spectrogram 	Audio File 7 Spectrogram 

Audio File 2 Spectrogram	Audio File 5 Spectrogram	Audio File 8 Spectrogram
		

I've used all 3 audio files of each category to extract an approximation to their respective watermark function. This was done by approximately finding the first and second peaks in the sine wave watermarks, and thus being able to calculate the wave's length/frequency. Here's a visual presentation of such peaks being found from file 3:



The full results are as seen in the following picture:

```
Files Categories:
Category 1:
  Files: ['Task 2/0_watermarked.wav', 'Task 2/1_watermarked.wav', 'Task 2/2_watermarked.wav']
  Real function: Approximately f ~= sin(2π·0.350) #Amplitude seems different for each file so we don't include it

Category 2:
  Files: ['Task 2/3_watermarked.wav', 'Task 2/4_watermarked.wav', 'Task 2/5_watermarked.wav']
  Real function: Approximately f ~= sin(2π·0.500) #Amplitude seems different for each file so we don't include it

Category 3:
  Files: ['Task 2/6_watermarked.wav', 'Task 2/7_watermarked.wav', 'Task 2/8_watermarked.wav']
  Real function: Approximately f ~= sin(2π·0.650) #Amplitude seems different for each file so we don't include it
```

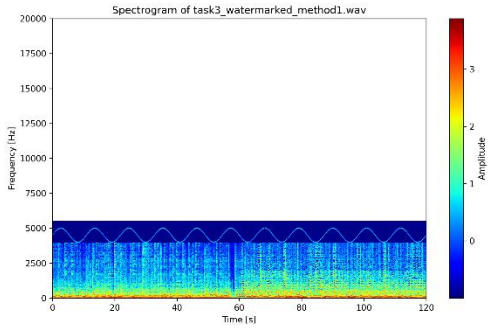
And to be more explicit, the functions are: **Category 1 ~= a*sin(2pi*0.350)**, **Category 2 ~= a*sin(2pi*0.5)** and finally **Category 3 ~= a*sin(2pi*0.65)**, with different amplitudes for each file that we can't exactly calculate so we don't write any amplitude explicitly.

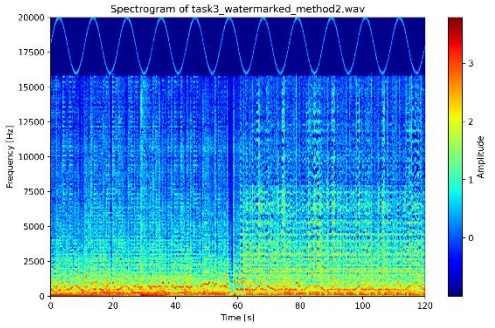
To implement that classification, what I did was read each audio file, truncate the audio

for the first 5 seconds as it looks like for each watermark, a full sine cycle completes by then, perform STFT, clip it's frequencies to be between 15kHz and 21kHz, filter out weak signals by amplitude strength, find the dominant frequency for each time slice (this is not always correct because there is "noise" or other frequencies that do not belong to the watermark in these frequencies levels, but it doesn't impact the algorithm heavily), detect the 1st peak before 1s as for all watermarks the highest spot of the sine waves comes before 1s, and then find the 2nd peak right after. We calculate the approximation of the sine wave cycle by calculating the distance between the peaks, and from there we can calculate an approximation for the sine wave frequency.
I've used the numpy and scipy libraries for calculations and algorithms implementations as these libraries are very known and easy to use, and we've used numpy in the past as well. As for plotting I've used matplotlib as it's very convenient and efficient for plotting, and I've used it in the past.
In order to remove the watermark, I would calculate the average amplitude of each frequency the watermark was present in, sample every x amount of time where the watermark should be based on the sine function we deduced it came from and remove the frequency corresponding to that function with relation to time, all in while it has an amplitude close enough to the average amplitude we calculated.

Determining speedup method

In this task there are 2 speedup/slowdown methods. One of them is in the frequency domain, meaning, as we learnt in class, we can for example - sample from the spectrogram of the original file every 'x' factor of slowdown/speedup sample, and reconstruct the audio. Frequencies of sound will remain unchanged. The second method, in the time domain, involves playing the file at a slower rate. 1 sec of sound will be played as 4 seconds.
These are the spectrograms of the task3_watermarked_method1.wave and task3_watermarked_method2.wave:





Since we've lost all frequencies above 5kHz in the **1st file**, we can deduce the method to slow it down was in the **Time Domain**. As we've learnt in class, if we speedup/slowdown in the frequency domain, frequencies of sound will remain unchanged and so the **2nd file** was slowed down in the **Frequency Domain.** The watermark hint also corresponds to the files from Category 1 in Task 2 and so it makes it easier for us to make these deductions based on the past information written.
As it looks like in the 1st spectrogram, we've lost all frequencies above 5kHz and so from that we will determine the **slowdown factor x = 4**. All in all, the original file is being played 4 times longer than it was, that is what I mean by slowdown.

Conclusion

In this exercise, we delved into the fascinating realm of audio signal processing, tackling three interconnected tasks that revolved around watermarking, classification, and speedup detection. Each task required a blend of theoretical understanding, practical implementation, and critical analysis to achieve meaningful results. Through the systematic application of tools such as Fourier Transforms, spectrogram analysis, and computational algorithms, we were able to design and evaluate effective solutions tailored to specific challenges.
The creation of good and bad watermarks highlighted the balance between subtlety and detectability in audio watermarking. The classification task emphasized the importance of precise frequency analysis and provided valuable insights into signal characteristics, such as sine wave patterns and their categorization. Finally, the speedup detection task showcased the impact of time-domain and frequency-domain manipulations on audio signals, offering a clear distinction between the methods based on spectrogram features.
Beyond the technical achievements, this exercise underscored the value of iterative experimentation and parameter tuning to refine results. The tools and techniques used are not only foundational for audio processing but also applicable to broader areas such as image analysis, media integrity verification, and communication systems. The skills and insights gained through this exercise provide a robust platform for tackling more complex challenges in digital signal processing and related fields.